

Research Report: C++ Implementation of Tracking Control Neural Network Simulations with Adversarial Attack Capabilities

Date: 2026-02-14

Author: Research Session (Ablasse Kingcaid-Ouedraogo, CAHSI REU)

Query: Comprehensive C++ implementation methods for tracking control neural network simulations with adversarial attack capabilities

Executive Summary

This report presents a comprehensive analysis of C++ implementation strategies for porting MATLAB-based sliding mode tracking control simulations to C++, with emphasis on neural network integration and adversarial attack capabilities. The research covers neural network inference frameworks (LibTorch, ONNX Runtime, RTNeural, TensorFlow Lite), control systems simulation libraries (Eigen, Control Toolbox, ToolboxR, Boost.odeint), adversarial attack implementation strategies (FGSM, PGD, FDI, sensor spoofing, PRBS injection), and performance optimization techniques (CUDA, TensorRT, vectorization). The primary recommendation is an architecture combining **Eigen** for linear algebra, **ONNX Runtime** or **LibTorch** for neural network inference, and custom C++ implementations for adversarial attacks and PRBS generation, targeting both desktop simulation and potential embedded deployment.

Table of Contents

- [1. Neural Network Libraries for C++](#)
- [2. Control Systems Simulation Frameworks](#)
- [3. Adversarial Attack Implementation](#)
- [4. MATLAB to C++ Migration Strategies](#)
- [5. Performance Optimization](#)
- [6. Academic & Industry References](#)
- [7. Library Comparison Matrix](#)
- [8. Architecture Recommendations](#)
- [9. Migration Roadmap](#)

- 10. [Code Examples](#)
 - 11. [Knowledge Gaps](#)
 - 12. [Sources](#)
-

1. Neural Network Libraries for C++

1.1 LibTorch (PyTorch C++ Frontend)

Overview: LibTorch is PyTorch's official C++ API, providing a pure C++17 interface that closely mirrors the Python API. It supports both model training and inference in C++.

Key Capabilities:

- Full `torch::nn` module for building networks from scratch in C++
- TorchScript model loading via `torch::jit::load()` for deploying Python-trained models
- CUDA support for GPU-accelerated inference
- Tensor operations matching MATLAB's matrix semantics
- Autograd for gradient computation (essential for adversarial attacks)

Deployment Path (Current Best Practice - AOTInductor):

As of PyTorch 2.10, TorchScript is **deprecated**. The recommended path is:

1. Train model in Python
2. Export with `torch.export.export()`
3. Compile with `torch._inductor.aoti_compile_and_package()` producing a `.pt2` shared library
4. Load in C++ using LibTorch's AOTInductor runtime
5. Build with CMake, setting `CMAKE_PREFIX_PATH` to the LibTorch installation

Strengths for This Project:

- Autograd enables gradient-based adversarial attacks (FGSM, PGD) directly in C++
- Rich tensor API simplifies porting MATLAB matrix operations
- Active development with AOTInductor as the future-proof deployment path
- CUDA support for GPU acceleration

Weaknesses:

- Large binary footprint (~500MB+ with CUDA)
- Not suitable for microcontroller/deeply embedded deployment

- AOTInductor workflow still maturing (inference inputs must match export-time shapes)

Version: PyTorch 2.10 / LibTorch 2.7.0 (latest stable with CUDA 11.8/12.x)

1.2 ONNX Runtime

Overview: Microsoft's cross-platform ML inference engine supporting models from PyTorch, TensorFlow, scikit-learn via the ONNX interchange format.

Key Capabilities:

- C and C++ APIs for model loading and inference
- Multiple execution providers: CPU, CUDA, TensorRT, DirectML, OpenVINO
- Sub-millisecond inference latency (0.98ms on RTX 2080 Ti with CUDA)
- Cross-platform: cloud, edge, mobile, web

C++ Inference Pattern:

```
// Environment and session setup
Ort::Env env(OrtLoggingLevel::ORT_LOGGING_LEVEL_WARNING, "control_nn");
Ort::SessionOptions sessionOptions;
sessionOptions.SetIntraOpNumThreads(1);
// Optional: Enable CUDA
// OrtSessionOptionsAppendExecutionProvider_CUDA(sessionOptions, 0);
Ort::Session session(env, "controller_model.onnx", sessionOptions);

// Create input tensor
Ort::MemoryInfo memInfo = Ort::MemoryInfo::CreateCpu(
    OrtArenaAllocator, OrtMemTypeDefault);
std::vector<float> inputData = {x1, x2, e1, e2, sigma};
std::vector<int64_t> inputShape = {1, 5};
Ort::Value inputTensor = Ort::Value::CreateTensor<float>(
    memInfo, inputData.data(), inputData.size(),
    inputShape.data(), inputShape.size());

// Run inference
const char* inputNames[] = {"state_input"};
const char* outputNames[] = {"control_output"};
auto outputTensors = session.Run(Ort::RunOptions{nullptr},
    inputNames, &inputTensor, 1, outputNames, 1);

// Extract output
float* controlOutput = outputTensors[0].GetTensorMutableData<float>();
```

Strengths for This Project:

- Framework-agnostic: train in PyTorch or TensorFlow, deploy via ONNX
- Lightweight compared to full LibTorch
- Excellent CPU inference performance with MKL-DNN
- Well-documented C++ API

Weaknesses:

- No autograd/gradient computation (cannot implement gradient-based attacks natively)
- Read-only inference only; cannot modify model weights at runtime
- ONNX export can lose some model semantics

Version: ONNX Runtime 1.20.x (latest)

1.3 RTNeural

Overview: A lightweight, header-only C++ neural network inference library designed for hard real-time constraints, originally targeting audio processing but applicable to any real-time control system.

Key Capabilities:

- Header-only library with minimal dependencies (Eigen, xsimd, or STL backend)
- Compile-time model architecture specification for maximum performance
- Supports Dense, GRU, LSTM, Conv1D, Conv2D, BatchNorm layers
- Activation functions: tanh, ReLU, Sigmoid, SoftMax, ELU, PReLU
- JSON model weight loading from PyTorch/TensorFlow exports

API Examples:

```
// Runtime model loading
#include <RTNeural.h>
std::ifstream jsonStream("controller_weights.json", std::ifstream::binary);
auto model = RTNeural::json_parser::parseJson<double>(jsonStream);
model->reset();
double input[] = {x_state, y_state, error};
double output = model->forward(input);

// Compile-time model (better performance)
RTNeural::ModelT<double, 3, 1,
    RTNeural::DenseT<double, 3, 16>,
    RTNeural::TanhActivationT<double, 16>,
    RTNeural::DenseT<double, 16, 16>,
    RTNeural::TanhActivationT<double, 16>,
    RTNeural::DenseT<double, 16, 1>
```

```
> controllerNN;  
// Load weights from JSON  
auto modelJson = nlohmann::json::parse(jsonStream);  
RTNeural::torch_helpers::loadDense(modelJson, "fc1.", controllerNN.get<0>()  
());
```

Strengths for This Project:

- Extremely low latency (microsecond-scale inference)
- No heap allocation during inference (real-time safe)
- Compile-time model specification eliminates virtual dispatch overhead
- Small binary footprint; ideal for embedded deployment
- Eigen backend provides good numerical accuracy

Weaknesses:

- Limited layer types (no custom layers for specialized control architectures)
- No training capability; inference only
- No GPU/CUDA support
- No autograd for adversarial attacks

Version: Latest release on GitHub (BSD-3-Clause license)

1.4 TensorFlow Lite (LiteRT)

Overview: Google's lightweight ML inference solution for mobile and embedded devices. Recently rebranded as LiteRT with version 2.x introducing the CompiledModel API.

Key Capabilities:

- C++ API via `tflite::Interpreter`
- Support for Cortex-M, RISC-V, ARC, Xtensa microcontrollers
- CMSIS-NN and vendor-specific kernel acceleration
- Model conversion from TensorFlow, PyTorch, JAX via `.tflite` format
- Quantization support (INT8, float16) for reduced memory/latency

Strengths for This Project:

- Best option for microcontroller deployment
- Extensive quantization tools for resource-constrained platforms
- Edge TPU acceleration available via Coral

Weaknesses:

- More complex C++ API compared to ONNX Runtime
- Primary ecosystem is TensorFlow (extra conversion step from PyTorch)
- Limited operator coverage compared to full TensorFlow

Version: LiteRT 2.x (formerly TensorFlow Lite)

1.5 MiniDNN

Overview: A header-only C++ library for feed-forward neural networks, implemented in C++98 with Eigen as its sole dependency.

Key Capabilities:

- Convolutional, max pooling, and fully connected layers
- Training and inference in pure C++
- RegressionMSE and classification loss functions
- RMSProp and other optimizers

Strengths for This Project:

- Simplest integration path; header-only with only Eigen dependency
- Can train networks directly in C++ (useful for online adaptation)
- Educational value for understanding DNN internals
- C++98 compatible (broadest compiler support)

Weaknesses:

- Limited layer types and features
- No GPU acceleration
- Not optimized for production use
- Small community and infrequent updates

1.6 tiny-cuda-nn (NVIDIA)

Overview: Lightning-fast C++/CUDA neural network framework from NVIDIA Labs with JIT compilation that fuses encodings, networks, loss functions, and backpropagation into single CUDA kernels.

Key Capabilities:

- 1.5x-2.5x faster inference than standard CUDA implementations
- Fully fused MLP support
- JSON configuration for network architecture

- Training and inference

Strengths: Maximum GPU performance for dense networks

Weaknesses: NVIDIA GPU required; not suitable for CPU-only deployment

2. Control Systems Simulation Frameworks

2.1 Eigen Linear Algebra Library

Overview: The de facto standard for linear algebra in C++. Header-only, providing highly efficient implementations of matrix operations, linear solvers, and decompositions.

Relevance to MATLAB Porting:

Eigen directly maps to MATLAB's matrix operations. The MATLAB simulation code uses operations like matrix multiplication, element-wise operations, and function evaluation that translate cleanly to Eigen.

State-Space Simulation with Eigen:

```
#include <Eigen/Dense>

class DiscreteTimeSystem {
    Eigen::VectorXd x; // State vector
    double dt;         // Time step

public:
    DiscreteTimeSystem(int stateDim, double timestep)
        : x(Eigen::VectorXd::Zero(stateDim)), dt(timestep) {}

    //  $x(k+1) = f(x(k)) + g(x(k)) * u(k)$ 
    void step(const Eigen::VectorXd& u) {
        Eigen::VectorXd f_val = computeF(x);
        Eigen::MatrixXd g_val = computeG(x);
        x = f_val + g_val * u;
    }

    // CSTR dynamics from fig2a.m
    Eigen::VectorXd computeF(const Eigen::VectorXd& state) {
        double x1 = state(0), x2 = state(1);
        double alpha = 1.0, Da = 0.072, gamma = 20.0, B = 1.0;

        Eigen::VectorXd f(2);
        double reaction = Da * (1.0 - x1) * std::exp(x2 / (1.0 + x2 /
```

```

gamma));
    f(0) = x1 + dt * (-alpha * x1 + reaction);
    f(1) = x2 + dt * (-alpha * x2 + B * reaction);
    return f;
}

Eigen::MatrixXd computeG(const Eigen::VectorXd& state) {
    Eigen::MatrixXd g = Eigen::MatrixXd::Zero(2, 2);
    g(1, 1) = 0.3; // lambda parameter
    return g;
}
};

```

Matrix Functions (Unsupported Module):

```

#include <unsupported/Eigen/MatrixFunctions>
// Matrix exponential for discretization: Ad = exp(A*dt)
Eigen::MatrixXd Ad = (A * dt).exp();

```

Version: Eigen 3.4.x (header-only, MPL2 license)

2.2 Control Toolbox (ETH Zurich)

Overview: An open-source C++ library for robotics, optimal control, and model predictive control, developed at ETH Zurich's Agile & Dexterous Robotics Lab (2014-2018).

Architecture:

- **ct_core:** Fundamental types (System, StateVector, Controller, Integrator)
- **ct_optcon:** Optimal control (iLQR, GNMS, DMS, MPC wrapper)
- **ct_rbd:** Rigid body dynamics
- **ct_models:** Sample robot models

ODE Integrators:

- Fixed step: Euler (`IntegratorEuler`), RK4 (`IntegratorRK4`)
- Variable step: RK5 (`IntegratorRK5Variable`), ODE45
- Symplectic (semi-implicit) integrators
- All integrators return `DiscreteTrajectory` objects

Relevance to This Project:

- Defines `ct::core::System` base class for dynamical systems
- Supports both continuous and discrete-time system definitions

- MPC wrapper `optcon::MPC` for receding-horizon control
- Sensitivity computation for linearized dynamics (Jacobians A_n , B_n)

Dependencies: Eigen, Kindr (header-only, Eigen-based). No ROS dependency required.

Limitation: Last active development was 2018. May require updates for modern C++ compilers.

2.3 ToolboxR (Discrete-Time Modeling Library)

Overview: A comprehensive C++ library specifically designed for discrete-time system modeling and control.

Key Components:

- **Discrete-Time Blocks:** Delay, Difference, Derivative, FIR/IIR filters
- **Butterworth Filters:** Lowpass, highpass, bandpass, notch
- **Discrete Integrators:** Trapezoidal, Forward/Backward Euler methods
- **PID Controller:** With anti-windup and output saturation
- **State-Space Representation:** Using Eigen library
- **Fuzzy Control Blocks:** Mamdani and Sugeno inference
- **Trajectory Generation:** Jerk-limited, quintic, heptic polynomials
- **Motor Models:** DC motor, PMSM with controllers
- **FFT Utilities:** Frequency response analysis

Strengths: Most directly applicable to the discrete-time sliding mode control problem. Can import MATLAB/Simulink FIS files.

Weaknesses: Work in progress; Visual Studio 2022 build system; MIT license.

2.4 Boost.odeint

Overview: Part of the Boost C++ Libraries, providing a generic framework for ODE integration with multiple stepper types.

Stepper Types:

- `euler` : First-order Euler method
- `runge_kutta4` : Classical 4th-order Runge-Kutta
- `runge_kutta_dopri5` : Dormand-Prince 5(4) adaptive stepper
- `rosenbrock4` : Implicit method for stiff systems

Usage Pattern:

```

#include <boost/numeric/odeint.hpp>
using namespace boost::numeric::odeint;

typedef std::vector<double> state_type;

// Define system dynamics
void cstr_system(const state_type& x, state_type& dxdt, double t) {
    double alpha = 1.0, Da = 0.072, gamma = 20.0, B = 1.0;
    double reaction = Da * (1.0 - x[0]) * std::exp(x[1] / (1.0 + x[1] /
gamma));
    dxdt[0] = -alpha * x[0] + reaction;
    dxdt[1] = -alpha * x[1] + B * reaction;
}

// Integrate
state_type x = {0.0, 0.0};
runge_kutta4<state_type> stepper;
double dt = 0.001;
for (double t = 0.0; t < 45.0; t += dt) {
    stepper.do_step(cstr_system, x, t, dt);
}

```

Note: For the discrete-time formulation in the MATLAB code (which already uses forward Euler), Boost.odeint may be overkill. Direct Eigen-based stepping is simpler and more appropriate.

3. Adversarial Attack Implementation

3.1 Fast Gradient Sign Method (FGSM)

Concept: Perturbs input along the gradient direction to maximize loss:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

C++ Implementation Strategy:

Option A: LibTorch with Autograd

```

#include <torch/torch.h>

torch::Tensor fgsm_attack(torch::jit::script::Module& model,
                          torch::Tensor input,
                          torch::Tensor target,

```

```

        float epsilon) {
    input.set_requires_grad(true);
    auto output = model.forward({input}).toTensor();
    auto loss = torch::mse_loss(output, target);
    loss.backward();

    auto perturbation = epsilon * input.grad().sign();
    auto adversarial_input = input + perturbation;
    return adversarial_input.detach();
}

```

Option B: Manual Gradient Estimation (No Autograd)

```

// Finite-difference gradient estimation for ONNX Runtime or RTNeural
Eigen::VectorXd estimate_gradient(
    std::function<double(const Eigen::VectorXd&)> loss_fn,
    const Eigen::VectorXd& x, double delta = 1e-5) {

    Eigen::VectorXd grad(x.size());
    for (int i = 0; i < x.size(); ++i) {
        Eigen::VectorXd x_plus = x, x_minus = x;
        x_plus(i) += delta;
        x_minus(i) -= delta;
        grad(i) = (loss_fn(x_plus) - loss_fn(x_minus)) / (2.0 * delta);
    }
    return grad;
}

Eigen::VectorXd fgsm_attack(
    std::function<double(const Eigen::VectorXd&)> loss_fn,
    const Eigen::VectorXd& x, double epsilon) {

    Eigen::VectorXd grad = estimate_gradient(loss_fn, x);
    return x + epsilon * grad.array().sign().matrix();
}

```

3.2 Projected Gradient Descent (PGD)

Concept: Iterative FGSM with projection back to epsilon-ball:

$$x_{k+1} = \Pi_{B_\epsilon(x)}(x_k + \alpha \cdot \text{sign}(\nabla_x J(\theta, x_k, y)))$$

```

Eigen::VectorXd pgd_attack(
    std::function<double(const Eigen::VectorXd&)> loss_fn,
    const Eigen::VectorXd& x_original,

```

```

double epsilon, double alpha, int num_steps) {

Eigen::VectorXd x_adv = x_original;
for (int step = 0; step < num_steps; ++step) {
    Eigen::VectorXd grad = estimate_gradient(loss_fn, x_adv);
    x_adv = x_adv + alpha * grad.array().sign().matrix();

    // Project back to epsilon-ball
    Eigen::VectorXd delta = x_adv - x_original;
    delta = delta.cwiseMax(-epsilon *
Eigen::VectorXd::Ones(delta.size()))
                .cwiseMin(epsilon *
Eigen::VectorXd::Ones(delta.size()));
    x_adv = x_original + delta;
}
return x_adv;
}

```

3.3 False Data Injection (FDI) Attack

Concept: Attacker injects additive bias into sensor measurements:

$$y_{attacked}(k) = y(k) + a(k)$$

where $a(k)$ is the injected false data designed to be stealthy (evade detection).

```

class FDIAttack {
    double attack_magnitude;
    int attack_start_step;
    int attack_end_step;
    bool stealthy; // Whether to match residual statistics

public:
    FDIAttack(double mag, int start, int end, bool stealth = true)
        : attack_magnitude(mag), attack_start_step(start),
          attack_end_step(end), stealthy(stealth) {}

    Eigen::VectorXd inject(const Eigen::VectorXd& measurement, int step) {
        if (step < attack_start_step || step > attack_end_step)
            return measurement;

        Eigen::VectorXd attacked = measurement;
        if (stealthy) {
            // Gradual ramp to avoid spike detection
            double ramp = std::min(1.0,
                static_cast<double>(step - attack_start_step) / 100.0);

```

```

        attacked(0) += attack_magnitude * ramp;
    } else {
        attacked(0) += attack_magnitude;
    }
    return attacked;
}
};

```

3.4 Sensor Spoofing

```

class SensorSpoofingAttack {
    enum class SpoofType { BIAS, SCALE, REPLAY, RANDOM };
    SpoofType type;
    double magnitude;
    std::vector<Eigen::VectorXd> replay_buffer;
    std::mt19937 rng;

public:
    Eigen::VectorXd spoof(const Eigen::VectorXd& true_measurement, int step)
    {
        switch (type) {
            case SpoofType::BIAS:
                return true_measurement.array() + magnitude;
            case SpoofType::SCALE:
                return true_measurement * (1.0 + magnitude);
            case SpoofType::REPLAY:
                if (step < replay_buffer.size())
                    return replay_buffer[step % replay_buffer.size()];
                return true_measurement;
            case SpoofType::RANDOM: {
                std::normal_distribution<double> dist(0.0, magnitude);
                Eigen::VectorXd noise(true_measurement.size());
                for (int i = 0; i < noise.size(); ++i)
                    noise(i) = dist(rng);
                return true_measurement + noise;
            }
        }
        return true_measurement;
    }
};

```

3.5 PRBS (Pseudo-Random Binary Sequence) Generation

Theory: PRBS is generated using a Linear Feedback Shift Register (LFSR). For PRBS-N, the sequence length is $2^N - 1$.

```

class PRBSGenerator {
    uint32_t lfsr;
    uint32_t polynomial;
    int nbits;
    double amplitude;

public:
    // Standard polynomials: PRBS7=0x41, PRBS9=0x110, PRBS11=0x500,
    // PRBS15=0x6000, PRBS23=0x420000, PRBS31=0x48000000
    PRBSGenerator(int n, double amp = 1.0) : nbits(n), amplitude(amp) {
        lfsr = 1; // Non-zero seed
        switch (n) {
            case 7: polynomial = 0x41; break; //  $x^7 + x^6 + 1$ 
            case 9: polynomial = 0x110; break; //  $x^9 + x^5 + 1$ 
            case 11: polynomial = 0x500; break; //  $x^{11} + x^9 + 1$ 
            case 15: polynomial = 0x6000; break; //  $x^{15} + x^{14} + 1$ 
            case 23: polynomial = 0x420000; break; //  $x^{23} + x^{18} + 1$ 
            case 31: polynomial = 0x48000000; break; //  $x^{31} + x^{28} + 1$ 
            default: polynomial = 0x41; nbits = 7;
        }
    }

    double next() {
        uint32_t feedback = 0;
        uint32_t masked = lfsr & polynomial;
        // Count parity of masked bits
        while (masked) {
            feedback ^= (masked & 1);
            masked >>= 1;
        }
        lfsr = (lfsr >> 1) | (feedback << (nbits - 1));
        return (lfsr & 1) ? amplitude : -amplitude;
    }

    // Generate signal for injection into control loop
    std::vector<double> generate(int length) {
        std::vector<double> signal(length);
        for (int i = 0; i < length; ++i)
            signal[i] = next();
        return signal;
    }
};

```

3.6 Probabilistic Speed Spoofing

```

class SpeedSpoofingAttack {
    double spoof_probability;
    double max_deviation;
    std::mt19937 rng;
    std::uniform_real_distribution<double> prob_dist{0.0, 1.0};
    std::normal_distribution<double> deviation_dist;

public:
    SpeedSpoofingAttack(double prob, double max_dev, unsigned seed = 42)
        : spoof_probability(prob), max_deviation(max_dev), rng(seed),
          deviation_dist(0.0, max_dev / 3.0) {}

    double spoof(double true_speed) {
        if (prob_dist(rng) < spoof_probability) {
            return true_speed + deviation_dist(rng);
        }
        return true_speed;
    }
};

```

3.7 Simulation Toolkits for CPS Security

CPSim (Python): A simulation toolbox for security problems in cyber-physical systems supporting bias attacks, delay attacks, and replay attacks. Compatible with external simulators and real testbeds. Uses state-space LTI models for linear benchmarks and ODEs for nonlinear benchmarks. Available at: <https://github.com/lion-zhang/CPSim>

4. MATLAB to C++ Migration Strategies

4.1 MATLAB Coder (Automatic Code Generation)

Capabilities:

- Generates generic ANSI/ISO C/C++ code from MATLAB functions
- Deep learning support: generates C++ for pretrained networks (no third-party dependencies)
- Intel MKL-DNN and ARM Compute Library acceleration
- Simulink Coder integration for model-based code generation

Workflow:

1. Annotate MATLAB code with `codegen` directives and type specifications

2. Use `codegen` command or MATLAB Coder app to generate C/C++
3. Validate generated code against MATLAB results
4. Integrate into C++ build system

Limitation for This Project: The current MATLAB code uses features (dynamic arrays, plotting) that require refactoring before code generation. MATLAB Coder works best with functions, not scripts.

4.2 Manual Porting Best Practices

Step 1: Refactor MATLAB Scripts to Functions

```
% Before: Script (fig2a.m)
x(1)=0; y(1)=0; n=45000; ...

% After: Function
function [t, x, y, e1, e2, u] = simulate_cstr(params)
    x = zeros(1, params.n);
    y = zeros(1, params.n);
    ...
end
```

Step 2: Create C++ Equivalents

Map MATLAB constructs to C++:

MATLAB	C++ (Eigen)
<code>x(i)</code>	<code>x(i-1)</code> (0-indexed)
<code>zeros(1,n)</code>	<code>Eigen::VectorXd::Zero(n)</code>
<code>exp(x)</code>	<code>std::exp(x)</code>
<code>sign(x)</code>	Custom: <code>(x > 0) - (x < 0)</code>
<code>randn(1)</code>	<code>std::normal_distribution<></code>
<code>A * B</code> (matrix)	<code>A * B</code> (Eigen)
<code>A .* B</code> (element-wise)	<code>A.cwiseProduct(B)</code>
<code>plot(t,x)</code>	Use <code>gnuplot-iostream</code> or <code>matplotlibcpp</code>

Step 3: Numerical Accuracy Verification

Key concerns when porting:

1. **Transcendental functions** (`exp` , `sin` , `cos`): IEEE-754-2008 does not require correct rounding. MATLAB and C++ implementations may differ at the ULP level.
2. **Compiler optimizations**: Different `-O` levels and floating-point modes (`-ffast-math`) can change results. Use `-ffp-contract=off` for strict IEEE compliance.
3. **Index translation**: MATLAB is 1-indexed; C++ is 0-indexed. Off-by-one errors are the most common bug.
4. **Sign function**: MATLAB's `sign(0) = 0` , which must be matched in C++.

Validation Strategy:

```
// Save MATLAB outputs to CSV, compare in C++
void validate_against_matlab(const std::string& matlab_csv,
                           const Eigen::VectorXd& cpp_result,
                           double tolerance = 1e-10) {
    auto matlab_data = loadCSV(matlab_csv);
    double max_error = (cpp_result - matlab_data).cwiseAbs().maxCoeff();
    assert(max_error < tolerance && "C++ result diverges from MATLAB");
}
```

4.3 Specific MATLAB-to-C++ Translations for This Project

CSTR Dynamics (fig2a.m):

```
// Direct translation of the CSTR simulation loop
struct CSTRParams {
    double alpha = 1.0;
    double beta = 100.0;
    double lambda = 0.3;
    double gamma = 20.0;
    double B = 1.0;
    double Da = 0.072;
    int n = 45000;
    double tmax = 45.0;
};

void simulate_cstr(const CSTRParams& p,
                  Eigen::VectorXd& x, Eigen::VectorXd& y,
                  Eigen::VectorXd& t, Eigen::VectorXd& u2) {
    double dt = p.tmax / p.n;
    x.setZero(p.n);
    y.setZero(p.n);
    t.setZero(p.n);
    u2.setZero(p.n);
}
```

```

Eigen::VectorXd r1(p.n), r2(p.n), e1(p.n), e2(p.n);
r1.setZero(); r2.setZero();
e1(0) = x(0) - r1(0);
e2(0) = y(0) - r2(0);

for (int i = 0; i < p.n - 1; ++i) {
    double g2 = p.lambda;

    double reaction = p.Da * (1.0 - x(i)) *
        std::exp(y(i) / (1.0 + y(i) / p.gamma));
    double f1 = x(i) + dt * (-p.alpha * x(i) + reaction);
    double f2 = y(i) + dt * (-p.alpha * y(i) + p.B * reaction);

    t(i+1) = t(i) + dt;

    // Piecewise reference
    if (t(i+1) >= 0.0 && t(i+1) <= 15.0) {
        r1(i+1) = 0.4472; r2(i+1) = 2.752;
    } else if (t(i+1) > 15.0 && t(i+1) < 30.0) {
        r1(i+1) = 0.7646; r2(i+1) = 4.7052;
    } else {
        r1(i+1) = 0.4472; r2(i+1) = 2.752;
    }

    // Sliding mode control
    double S;
    if (y(i) - r2(i) > 0) S = 1.0;
    else if (y(i) - r2(i) < 0) S = -1.0;
    else S = 0.0;

    u2(i) = -1.0 / g2 * (-p.betta * S + r2(i+1) - r2(i) - f2);

    x(i+1) = f1;
    y(i+1) = y(i) + dt * (-p.betta * S + r2(i+1) - r2(i));

    e1(i+1) = x(i+1) - r1(i+1);
    e2(i+1) = y(i+1) - r2(i+1);
}
}

```

5. Performance Optimization

5.1 Expected Speedups

Based on published benchmarks:

Scenario	MATLAB	C++ (CPU)	C++ (CUDA)
Loop-heavy simulation (45k steps)	1x (baseline)	5-50x faster	10-100x faster
Matrix operations (BLAS-dominated)	~1x (MKL)	~1x (MKL)	2-10x (cuBLAS)
NN inference (small dense network)	1x	10-100x	100-1000x
Chaotic system simulation	1x	1.26-1.54x	N/A
Large-scale power flow	1x	2.5x	N/A

Key insight: The MATLAB code in this project is loop-heavy with scalar operations (not vectorized matrix operations), which is precisely where C++ provides the largest speedup advantage.

5.2 CUDA/GPU Acceleration

For Neural Network Inference:

- **TensorRT:** Up to 40x faster than CPU-only. Optimizes graph, fuses layers, applies mixed precision.
- **cuDNN:** Foundation library for DNN primitives (convolution, GEMM, activation)
- **tiny-cuda-nn:** JIT compilation fuses entire MLP into single kernels (1.5-2.5x over standard CUDA)

For Control Simulation:

GPU acceleration of the simulation loop itself is **not recommended** for this project because:

1. The simulation is sequential (each step depends on the previous)
2. State vectors are small (2-3 elements)
3. GPU kernel launch overhead (~10us) exceeds the computation time per step

Recommended approach: Run the simulation loop on CPU, offload only NN inference to GPU if the network is large enough to justify transfer overhead.

5.3 Vectorization with Eigen

```
// Eigen automatically uses SIMD (SSE2/AVX/AVX2) for operations
// Compile with: -march=native -O3
Eigen::VectorXd x = Eigen::VectorXd::Random(1000);
Eigen::VectorXd y = x.array().exp(); // Vectorized exp()

// For Monte Carlo attack simulations, batch multiple trajectories
```

```
Eigen::MatrixXd X(state_dim, num_trajectories); // Column-major
// All trajectories advance in parallel via SIMD
```

5.4 Profiling Tools

Tool	Purpose	Platform
<code>perf</code>	CPU performance counters	Linux
<code>gprof</code>	Function-level profiling	Linux/macOS
Instruments (Xcode)	Time Profiler, Allocations	macOS
NVIDIA Nsight Systems	GPU timeline, kernel analysis	NVIDIA GPU
NVIDIA Nsight Compute	Kernel-level profiling	NVIDIA GPU
Valgrind/Cachegrind	Cache miss analysis	Linux
Intel VTune	Vectorization, threading	Intel CPU
Google Benchmark	Microbenchmarking framework	Cross-platform

5.5 Memory Management for Embedded

```
// Pre-allocate all buffers at initialization (no heap allocation in control
loop)
struct SimulationBuffers {
    Eigen::VectorXd state;
    Eigen::VectorXd control;
    Eigen::VectorXd reference;
    Eigen::VectorXd error;
    Eigen::VectorXd nn_input;
    Eigen::VectorXd nn_output;

    SimulationBuffers(int state_dim, int control_dim, int nn_in, int nn_out)
        : state(state_dim), control(control_dim),
          reference(state_dim), error(state_dim),
          nn_input(nn_in), nn_output(nn_out) {
        state.setZero();
        control.setZero();
        reference.setZero();
        error.setZero();
    }
};
```

6. Academic & Industry References

6.1 Neural Network Sliding Mode Control

Paper: "Neural network sliding mode control of multi-agent systems based on a preview mechanism and whale optimization algorithm"

- **Authors:** Yuxin Chen, Junchao Ren
- **Year:** 2025
- **Source:** Proceedings of the Institution of Mechanical Engineers, Part I
- **DOI:** 10.1177/09596518241302481
- **Relevance:** 5/5 - Directly addresses neural network sliding mode control for discrete-time multi-agent systems
- **Key Finding:** Combines NN-based SMC with whale optimization for unknown nonlinearities in discrete-time MAS

Paper: "Data-driven sliding mode tracking control with improved prescribed performance for a class of nonlinear discrete-time systems under DoS attacks and sensor faults"

- **Year:** 2024
- **Source:** International Journal of Dynamics and Control
- **DOI:** 10.1007/s40435-024-01517-1
- **Relevance:** 5/5 - Addresses tracking control with SMC under attacks for discrete-time nonlinear systems
- **Key Finding:** Uses RBF neural network for fault estimation in sliding mode tracking control under DoS attacks

Paper: "Adaptive sliding-mode tracking control of networked control systems with false data injection attacks"

- **Source:** Information Sciences (ScienceDirect)
- **DOI:** 10.1016/S0020-0255(21)01191-9
- **Relevance:** 5/5 - FDI attacks on sliding mode tracking control systems

Paper: "Sliding Mode Consensus Tracking Control for Multi-Agent Systems Under Hybrid Cyber Attacks Based on Dynamic Event-Triggered"

- **Authors:** Han et al.
- **Year:** 2025
- **Source:** International Journal of Adaptive Control and Signal Processing
- **DOI:** 10.1002/acs.4059
- **Relevance:** 4/5 - SMC under hybrid cyber attacks (deception + DoS)

6.2 Adversarial Attacks and Neural Network Robustness

Paper: "Adversarial Robustness of Deep Neural Networks: A Survey"

- **Year:** 2022 (arXiv: 2206.12227)
- **Relevance:** 4/5 - Comprehensive survey of FGSM, PGD, DeepFool, C&W attacks

Paper: "A Cost-Aware Approach to Adversarial Robustness in Neural Networks"

- **Year:** 2024 (arXiv: 2409.07609)
- **Relevance:** 3/5 - Modern approach to adversarial robustness

Paper: "Concealed Adversarial attacks on neural networks for sequential data"

- **Year:** 2025 (arXiv: 2502.20948)
- **Relevance:** 4/5 - Adversarial attacks on time-series models including RNNs and state-space models

Tool: alpha-beta-CROWN Neural Network Verifier

- Winner of VNN-COMP 2021-2025
- Can verify Lyapunov stability of NN controllers
- GPU-accelerated verification
- GitHub: <https://github.com/Verified-Intelligence/alpha-beta-CROWN>

6.3 False Data Injection in Control Systems

Paper: "False data injection attacks in control systems"

- **Authors:** Yilin Mo et al.
- **Source:** First Workshop on Secure Control Systems (2010, foundational)
- **Relevance:** 5/5 - Foundational paper on FDI attacks

Paper: "Data-driven Control Against False Data Injection Attacks"

- **Year:** 2023 (arXiv: 2311.08207)
- **Relevance:** 4/5 - Data-driven controllers for unknown linear systems under FDI

Paper: "Model-free False Data Injection Attack in Networked Control Systems"

- **Year:** 2022 (arXiv: 2212.07633)
- **Relevance:** 4/5 - Model-free FDI for general nonlinear systems

6.4 C++ Control Libraries

Paper: "The Control Toolbox - An Open-Source C++ Library for Robotics, Optimal and Model Predictive Control"

- **Year:** 2018 (arXiv: 1801.04290)
- **Authors:** ETH Zurich ADRL
- **Relevance:** 4/5 - Reference architecture for C++ control systems

Paper: "RTNeural: Fast Neural Inferencing for Real-Time Systems"

- **Year:** 2021 (arXiv: 2106.03037)
- **Relevance:** 4/5 - Benchmark results for real-time NN inference in C++

7. Library Comparison Matrix

Neural Network Inference Libraries

Feature	LibTorch	ONNX Runtime	RTNeural	TF Lite	MiniDNN	tiny-cuda-nn
Inference	Yes	Yes	Yes	Yes	Yes	Yes
Training	Yes	No	No	No	Yes	Yes
Autograd	Yes	No	No	No	No	Yes
CUDA/GPU	Yes	Yes	No	Edge TPU	No	Yes (required)
Header-only	No	No	Yes	No	Yes	No
Binary size	~500MB	~50MB	<1MB	~5MB	<100KB	~10MB
Latency	~1ms	~0.1-1ms	~1-10us	~0.5-5ms	~10us	~10us
C++ standard	C++17	C++14	C++14	C++11	C++98	C++17/CUDA
Model format	.pt2/TorchScript	.onnx	JSON	.tflite	Custom	JSON
Adversarial attacks	Native (autograd)	Manual only	Manual only	Manual only	Manual only	Native
Embedded suitable	No	Partial	Yes	Yes	Yes	No

Feature	LibTorch	ONNX Runtime	RTNeural	TF Lite	MiniDNN	tiny-cuda-nn
License	BSD	MIT	BSD-3	Apache-2	MPL-2	BSD-3

Control Simulation Libraries

Feature	Eigen	Control Toolbox	ToolboxR	Boost.odeint
Matrix ops	Full	Via Eigen	Via Eigen	Via state_type
ODE solvers	Manual	Euler/RK4/RK5/ODE45	Euler/Trapezoidal	Euler/RK4/RK5/Rosenl
State-space	Manual	ct::core::System	Built-in class	Manual
Discrete-time	Manual	Yes (sensitivity)	Native focus	Via stepper
MPC support	No	Yes <code>optcon</code>	No	No
PID control	No	No	Yes (with anti-windup)	No
Fuzzy control	No	No	Yes	No
Header-only	Yes	Partial	No	Yes (Boost)
Active maintenance	Yes	No (2018)	WIP	Yes

8. Architecture Recommendations

8.1 Recommended C++ Project Structure

```
tracking-control-network/
├── CMakeLists.txt
├── include/
│   ├── core/
│   │   ├── types.hpp           # State, Control, Reference types
│   │   ├── system_base.hpp     # Abstract base for dynamical systems
│   │   └── integrator.hpp      # Discrete-time stepping
│   └── systems/
│       ├── cstr_system.hpp     # CSTR reactor (fig2)
│       └── mechanical_system.hpp # Mechanical system (fig5)
```


- └─ controllers/
 - └─ sliding_mode.hpp # Classical SMC
 - └─ nn_controller.hpp # NN-based controller interface
 - └─ nn_smc.hpp # NN sliding mode controller
- └─ attacks/
 - └─ attack_base.hpp # Abstract attack interface
 - └─ fgsm.hpp # FGSM attack
 - └─ pgd.hpp # PGD attack
 - └─ fdi.hpp # False data injection
 - └─ sensor_spoofing.hpp # Sensor spoofing variants
 - └─ prbs_generator.hpp # PRBS noise injection
 - └─ speed_spoofing.hpp # Probabilistic speed spoofing
- └─ nn/
 - └─ inference_engine.hpp # Abstract NN inference interface
 - └─ onnx_engine.hpp # ONNX Runtime backend
 - └─ libtorch_engine.hpp # LibTorch backend
 - └─ rtneural_engine.hpp # RTNeural backend
- └─ simulation/
 - └─ simulator.hpp # Main simulation loop
 - └─ data_logger.hpp # CSV/binary data output
 - └─ reference_generator.hpp # Reference trajectory generation
- └─ src/
 - └─ main.cpp
 - └─ systems/
 - └─ controllers/
 - └─ attacks/
 - └─ nn/
 - └─ simulation/
- └─ models/ # Trained NN model files
 - └─ controller.onnx
 - └─ controller_weights.json
- └─ config/ # Simulation configuration
 - └─ simulation_params.yaml
- └─ tests/
 - └─ test_cstr.cpp
 - └─ test_attacks.cpp
 - └─ matlab_validation/ # MATLAB reference outputs
 - └─ fig2a_reference.csv
 - └─ fig5a_reference.csv
- └─ scripts/
 - └─ export_model.py # PyTorch → ONNX export
 - └─ validate_results.py # Cross-validation with MATLAB
- └─ third_party/
 - └─ eigen/
 - └─ rtneural/
 - └─ nlohmann_json/

8.2 Framework Selection Decision Tree

```
Is GPU available?
├─ YES: Is the NN large (>1M parameters)?
│   ├── YES: Use LibTorch or ONNX Runtime + CUDA
│   └── NO: Use ONNX Runtime (CPU) or RTNeural
└─ NO: Is embedded deployment required?
    ├── YES: Is microcontroller target?
    │   ├── YES: Use TF Lite Micro
    │   └── NO: Use RTNeural (header-only, Eigen backend)
    └── NO: Need gradient-based attacks?
        ├── YES: Use LibTorch (autograd support)
        └── NO: Use ONNX Runtime (best inference performance)
```

8.3 Primary Recommendation

For this CAHSI REU project, the recommended stack is:

Component	Choice	Rationale
Linear Algebra	Eigen 3.4	Industry standard, header-only, maps to MATLAB semantics
NN Inference (primary)	ONNX Runtime	Framework-agnostic, good C++ API, optional CUDA
NN Inference (embedded)	RTNeural	Header-only, real-time safe, microsecond latency
Adversarial Attacks	Custom C++ (with Eigen)	No existing C++ attack library; implement FGSM/PGD/FDI manually
Build System	CMake 3.20+	Standard for C++ projects with multiple dependencies
Testing	Google Test + MATLAB validation	Unit tests + cross-validation against MATLAB outputs
Data Logging	CSV + binary	For plotting in Python/MATLAB
Configuration	YAML (yaml-cpp) or JSON <code>json</code>	Human-readable simulation parameters

9. Migration Roadmap

Phase 1: Foundation (Weeks 1-2)

1. Set up C++ project structure with CMake
2. Integrate Eigen (header-only, trivial)
3. Implement `DiscreteTimeSystem` base class
4. Port CSTR dynamics (fig2a) to C++ with Eigen
5. Validate against MATLAB output (CSV comparison)

Phase 2: Control (Weeks 3-4)

6. Implement classical sliding mode controller in C++
7. Port fig2a complete simulation (dynamics + controller + reference)
8. Port fig5a mechanical system simulation
9. Add data logging (CSV output for post-processing in Python)
10. Cross-validate all results against MATLAB

Phase 3: Neural Network Integration (Weeks 5-6)

11. Train NN controller in Python (PyTorch)
12. Export to ONNX format
13. Integrate ONNX Runtime into C++ project
14. Create `NNController` class wrapping ONNX inference
15. Replace classical SMC with NN-SMC in simulation

Phase 4: Adversarial Attacks (Weeks 7-8)

16. Implement FGSM attack (finite-difference gradient)
17. Implement PGD attack (iterative FGSM)
18. Implement FDI attack module
19. Implement PRBS generator
20. Implement sensor spoofing variants
21. Run attack simulations, collect results

Phase 5: Optimization & Analysis (Weeks 9-10)

22. Profile C++ simulation performance
23. Add CUDA support for NN inference (if GPU available)
24. Benchmark C++ vs MATLAB execution times
25. Run comprehensive attack scenarios
26. Generate publication-quality plots

Phase 6: Documentation & Deployment (Weeks 11-12)

27. Write technical documentation
 28. Create reproducible benchmarks
 29. Package for potential embedded deployment
 30. Prepare research paper contributions
-

10. Code Examples

10.1 Complete Minimal Simulation (CSTR with SMC)

```
// main_cstr_smc.cpp
#include <Eigen/Dense>
#include <iostream>
#include <fstream>
#include <cmath>

struct CSTRParams {
    double alpha = 1.0, betta = 100.0, lambda = 0.3;
    double gamma = 20.0, B = 1.0, Da = 0.072;
    int n = 45000;
    double tmax = 45.0;
};

double sign_fn(double x) {
    if (x > 0.0) return 1.0;
    if (x < 0.0) return -1.0;
    return 0.0;
}

void get_reference(double t, double& r1, double& r2) {
    if (t >= 0.0 && t <= 15.0) {
        r1 = 0.4472; r2 = 2.752;
    } else if (t > 15.0 && t < 30.0) {
        r1 = 0.7646; r2 = 4.7052;
    } else {
        r1 = 0.4472; r2 = 2.752;
    }
}

int main() {
    CSTRParams p;
    double dt = p.tmax / p.n;

    Eigen::VectorXd x(p.n), y(p.n), t(p.n);
```

```

Eigen::VectorXd r1(p.n), r2(p.n), e1(p.n), e2(p.n), u2(p.n);

x(0) = 0.0; y(0) = 0.0; t(0) = 0.0;
r1(0) = 0.0; r2(0) = 0.0;
e1(0) = x(0) - r1(0);
e2(0) = y(0) - r2(0);
u2(0) = 0.0;

for (int i = 0; i < p.n - 1; ++i) {
    double g2 = p.lambda;
    double reaction = p.Da * (1.0 - x(i)) *
        std::exp(y(i) / (1.0 + y(i) / p.gamma));

    double f1 = x(i) + dt * (-p.alpha * x(i) + reaction);
    double f2 = y(i) + dt * (-p.alpha * y(i) + p.B * reaction);

    t(i+1) = t(i) + dt;
    get_reference(t(i+1), r1(i+1), r2(i+1));

    double S = sign_fn(y(i) - r2(i));
    u2(i) = -1.0/g2 * (-p.betta * S + r2(i+1) - r2(i) - f2);

    x(i+1) = f1;
    y(i+1) = y(i) + dt * (-p.betta * S + r2(i+1) - r2(i));

    e1(i+1) = x(i+1) - r1(i+1);
    e2(i+1) = y(i+1) - r2(i+1);
}

// Save to CSV
std::ofstream file("cstr_output.csv");
file << "t,x,y,r1,r2,e1,e2\n";
for (int i = 0; i < p.n; ++i) {
    file << t(i) << "," << x(i) << "," << y(i) << ","
        << r1(i) << "," << r2(i) << ","
        << e1(i) << "," << e2(i) << "\n";
}

std::cout << "Simulation complete. Output saved to cstr_output.csv\n";
std::cout << "Final state: x=" << x(p.n-1) << " y=" << y(p.n-1) << "\n";
return 0;
}

```

10.2 NN Controller with ONNX Runtime

```

// nn_controller_onnx.hpp
#pragma once
#include <onnxruntime_cxx_api.h>
#include <vector>
#include <string>

class NNController {
    Ort::Env env_;
    Ort::Session session_;
    Ort::MemoryInfo mem_info_;
    std::vector<const char*> input_names_;
    std::vector<const char*> output_names_;
    std::vector<int64_t> input_shape_;
    std::vector<int64_t> output_shape_;

public:
    NNController(const std::string& model_path)
        : env_(ORT_LOGGING_LEVEL_WARNING, "NNController"),
          session_(env_, model_path.c_str(), Ort::SessionOptions{}),
          mem_info_(Ort::MemoryInfo::CreateCpu(OrtArenaAllocator,
                                                OrtMemTypeDefault)) {

        // Query model metadata
        auto input_info = session_.GetInputTypeInfo(0);
        auto shape = input_info.GetTensorTypeAndShapeInfo().GetShape();
        input_shape_ = shape;

        auto output_info = session_.GetOutputTypeInfo(0);
        output_shape_ = output_info.GetTensorTypeAndShapeInfo().GetShape();

        input_names_ = {"input"};
        output_names_ = {"output"};
    }

    std::vector<float> compute_control(const std::vector<float>& state) {
        auto input_tensor = Ort::Value::CreateTensor<float>(
            mem_info_, const_cast<float*>(state.data()), state.size(),
            input_shape_.data(), input_shape_.size());

        auto output_tensors = session_.Run(
            Ort::RunOptions{nullptr},
            input_names_.data(), &input_tensor, 1,
            output_names_.data(), 1);

        float* output_data = output_tensors[0].GetTensorMutableData<float>
();
        int output_size = 1;
    }

```

```

        for (auto dim : output_shape_) output_size *= dim;

        return std::vector<float>(output_data, output_data + output_size);
    }
};

```

10.3 Attack Simulation Framework

```

// attack_simulation.hpp
#pragma once
#include <Eigen/Dense>
#include <memory>
#include <vector>

class AttackBase {
public:
    virtual ~AttackBase() = default;
    virtual Eigen::VectorXd apply(const Eigen::VectorXd& measurement,
                                  int timestep) = 0;
    virtual std::string name() const = 0;
    virtual bool is_active(int timestep) const = 0;
};

class AttackSimulator {
    std::vector<std::shared_ptr<AttackBase>> attacks_;

public:
    void add_attack(std::shared_ptr<AttackBase> attack) {
        attacks_.push_back(std::move(attack));
    }

    Eigen::VectorXd apply_attacks(const Eigen::VectorXd& measurement,
                                   int timestep) {
        Eigen::VectorXd result = measurement;
        for (auto& attack : attacks_) {
            if (attack->is_active(timestep)) {
                result = attack->apply(result, timestep);
            }
        }
        return result;
    }
};

// Usage in simulation loop:
// AttackSimulator attacker;

```

```

// attacker.add_attack(std::make_shared<FDIAAttack>(0.5, 1000, 3000));
// attacker.add_attack(std::make_shared<PRBSNoiseAttack>(0.1, 7));
//
// for (int k = 0; k < N; ++k) {
//     Eigen::VectorXd y_measured = get_measurement(state);
//     Eigen::VectorXd y_attacked = attacker.apply_attacks(y_measured, k);
//     Eigen::VectorXd u = controller.compute(y_attacked);
//     state = system.step(state, u);
// }

```

10.4 CMakeLists.txt

```

cmake_minimum_required(VERSION 3.20)
project(TrackingControlNN VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Eigen (header-only)
find_package(Eigen3 3.4 REQUIRED NO_MODULE)

# ONNX Runtime (optional)
option(USE_ONNX_RUNTIME "Enable ONNX Runtime for NN inference" ON)
if(USE_ONNX_RUNTIME)
    find_package(onnxruntime REQUIRED)
    add_definitions(-DUSE_ONNX_RUNTIME)
endif()

# RTNeural (optional, header-only)
option(USE_RTNEURAL "Enable RTNeural for lightweight NN inference" OFF)
if(USE_RTNEURAL)
    add_subdirectory(third_party/rtn neural)
    add_definitions(-DUSE_RTNEURAL)
endif()

# LibTorch (optional)
option(USE_LIBTORCH "Enable LibTorch for NN with autograd" OFF)
if(USE_LIBTORCH)
    find_package(Torch REQUIRED)
    add_definitions(-DUSE_LIBTORCH)
endif()

# Main executable
add_executable(tracking_sim
    src/main.cpp

```



```

src/systems/cstr_system.cpp
src/systems/mechanical_system.cpp
src/controllers/sliding_mode.cpp
src/attacks/fgsm.cpp
src/attacks/pgd.cpp
src/attacks/fdi.cpp
src/attacks/prbs_generator.cpp
src/attacks/sensor_spoofing.cpp
src/simulation/simulator.cpp
src/simulation/data_logger.cpp
)

target_include_directories(tracking_sim PRIVATE include)
target_link_libraries(tracking_sim PRIVATE Eigen3::Eigen)

if(USE_ONNX_RUNTIME)
    target_link_libraries(tracking_sim PRIVATE onnxruntime::onnxruntime)
endif()
if(USE_LIBTORCH)
    target_link_libraries(tracking_sim PRIVATE ${TORCH_LIBRARIES})
endif()

# Enable optimizations
target_compile_options(tracking_sim PRIVATE
    $<$<CONFIG:Release>:-O3 -march=native -DNDEBUG>
)

# Tests
option(BUILD_TESTS "Build unit tests" ON)
if(BUILD_TESTS)
    enable_testing()
    find_package(GTest REQUIRED)
    add_executable(test_simulation
        tests/test_cstr.cpp
        tests/test_attacks.cpp
    )
    target_link_libraries(test_simulation PRIVATE
        GTest::GTest GTest::Main Eigen3::Eigen)
    gtest_discover_tests(test_simulation)
endif()

```

11. Knowledge Gaps

Not Found / Requires Further Research

1. **No dedicated C++ adversarial attack library exists.** All existing attack implementations (torchattacks, CleverHans, ART) are Python-only. C++ attacks must be implemented from scratch.
 2. **Limited C++ benchmarks for NN-based control loops.** Most published benchmarks compare MATLAB vs C++ for general numerical computation, not specifically for control loop simulations with NN inference in the loop.
 3. **No papers found on C++ implementation of NN-SMC specifically.** While many papers address the theory of NN-based sliding mode controllers, implementation details typically reference MATLAB/Simulink, not C++.
 4. **DeepFool attack in C++.** This attack requires SVD computation at each iteration, making it computationally expensive. No C++ implementation was found; would need to be built using Eigen's SVD decomposition.
 5. **Real-time guarantees for NN inference in control loops.** While RTNeural provides real-time-safe inference, formal worst-case execution time (WCET) analysis for NN inference in safety-critical control is still an active research area.
 6. **MATLAB Coder effectiveness for this specific codebase.** The current MATLAB code uses scripts (not functions), dynamic arrays, and plotting commands that would need refactoring before MATLAB Coder could generate usable C++.
 7. **Comparative study of NN inference frameworks for small dense networks** (typical controller networks with <10K parameters). Most benchmarks focus on large vision models.
-

12. Sources

Official Documentation

- [PyTorch C++ Frontend Documentation](#)
- [PyTorch C++ Frontend Tutorial](#)
- [PyTorch AOTInductor \(C++ Deployment\)](#)
- [ONNX Runtime C++ Getting Started](#)
- [ONNX Runtime C API Reference](#)
- [ONNX Runtime Inference API Basics](#)
- [Eigen Matrix Functions \(unsupported\)](#)
- [Boost.odeint Steppers](#)
- [MATLAB Coder for Deep Learning](#)
- [MATLAB Coder C++ Code Generation](#)
- [LiteRT \(TensorFlow Lite\) Overview](#)

- [LiteRT for Microcontrollers](#)
- [cuDNN Library](#)
- [TensorRT Inference](#)
- [MATLAB Generated Code Differences](#)

GitHub Repositories

- [RTNeural - Real-time NN Inference](#)
- [MiniDNN - Header-only DNN Library](#)
- [tiny-cuda-nn - NVIDIA Fast NN Framework](#)
- [Control Toolbox - ETH Zurich](#)
- [ToolboxR - Discrete Time Modeling](#)
- [PRBS Generator \(C/Verilog\)](#)
- [Eigen State-Space Simulation Tutorial](#)
- [ONNX Runtime Inference C++ Example](#)
- [AOTInductor C++ Example](#)
- [ONNX Runtime Official](#)
- [ONNX Runtime Inference Examples](#)
- [PyTorch C++ Examples](#)
- [alpha-beta-CROWN NN Verifier](#)
- [CPSim - CPS Security Simulation](#)
- [torchattacks \(Python, reference\)](#)

Academic Papers

- Chen & Ren (2025). "Neural network sliding mode control of multi-agent systems based on a preview mechanism and whale optimization algorithm." *Proceedings of the Institution of Mechanical Engineers, Part I*. DOI: 10.1177/09596518241302481
- Han et al. (2025). "Sliding Mode Consensus Tracking Control for Multi-Agent Systems Under Hybrid Cyber Attacks." *Intl. J. Adaptive Control and Signal Processing*. DOI: 10.1002/acs.4059
- (2024). "Data-driven sliding mode tracking control with improved prescribed performance for nonlinear discrete-time systems under DoS attacks." *Intl. J. Dynamics and Control*. DOI: 10.1007/s40435-024-01517-1
- Mo et al. (2010). "False data injection attacks in control systems." *First Workshop on Secure Control Systems*.
- (2023). "Data-driven Control Against False Data Injection Attacks." arXiv: 2311.08207
- (2022). "Model-free False Data Injection Attack in Networked Control Systems." arXiv: 2212.07633

- Chowdhury (2021). "RTNeural: Fast Neural Inferencing for Real-Time Systems." arXiv: 2106.03037
- Gifftthaler et al. (2018). "The Control Toolbox." arXiv: 1801.04290
- (2022). "Adversarial Robustness of Deep Neural Networks: A Survey." arXiv: 2206.12227
- (2024). "A Cost-Aware Approach to Adversarial Robustness in Neural Networks." arXiv: 2409.07609
- (2025). "Concealed Adversarial attacks on neural networks for sequential data." arXiv: 2502.20948
- (2024). "CPSim: Simulation Toolbox for Security Problems in Cyber-Physical Systems." *ACM Trans. Design Automation of Electronic Systems*. DOI: 10.1145/3674904

Blog Posts & Tutorials

- [Simulating State-Space Models in Eigen C++](#)
- [ONNX Runtime C++ Inference Guide](#)
- [Floating-point agreement between MATLAB and C++](#)
- [Deploying PyTorch Models in C++](#)
- [Boosting ODE simulations with Boost.odeint](#)
- [Deep Learning on Microcontrollers: State of Embedded ML in 2025](#)

Report generated 2026-02-14 for CAHSI REU Research Project at Texas A&M University - Victoria